Trabajo práctico Nro 7

Diseño de Sistemas de Instrumentación. Medición de interferencias.

Filsinger Gonzalo, Perea Ignacio, Leon Parfait Manuel

Medidas Electrónicas 4R2, Ingeniería Electrónica, Facultad Regional Córdoba
Universidad Tecnológica Nacional

I. Integrantes y Fechas

Apellido y Nombre	Coordinador	Operador	Documentador
Perea Ignacio			X
Filsinger Gonzalo	X		
Leon Parfait Manuel		X	
Fecha de Presentación:		10/09/2026	

II. Experimento 1

Para este experimento haremos uso de nuestra propia computadora, la cual conectaremos al generador de funciones para controlarlo mediante un script en Python. Para ello primero deberemos realizar las siguientes cosas:

Se instalo la librería picocom, con la cual podremos comunicarnos con el generador de funciones mediante un puerto serial.

Luego teniendo el generador encendido y conectado a la computadora mediante un cable USB, abrimos una terminal y ejecutamos el siguiente comando:

```
sudo picocom -b 9600 /dev/ttyUSB0
```

Con dicho comando nos conectamos al generador de funciones mediante el puerto serial, y podremos enviarle comandos para controlarlo.

En un primer paso le mandaremos el comando *IDN?* para que nos devuelva el nombre del generador de funciones, y así verificar que la comunicación se haya establecido correctamente.

```
~/Medidas via  v3.14.4 took 2s
❯ sudo picocom -b 9600 -d 8 -p n /dev/ttyUSB0
picocom v3.1

port is        : /dev/ttyUSB0
flowcontrol    : none
baudrate is    : 9600
parity is      : none
databits are   : 8
stopbits are   : 1
escape is      : C-a
local echo is  : no
noinit is      : no
noreset is     : no
hangup is      : no
nolock is      : no
send_cmd is    : sz -vv
receive_cmd is : rz -vv -E
imap is        :
omap is        :
emap is        : crcrlf,delbs,
logfile is     : none
initstring     : none
exit_after is  : not set
exit is        : no

Type [C-a] [C-h] to see available commands
Terminal ready
15MHz, Function Generator, V.1.21
```

Figura 1. Respuesta del generador de funciones al comando *IDN?*

Como se puede observar en la figura, el generador de funciones nos devuelve sus datos y el ancho de banda, lo cual nos indica que la comunicación se estableció correctamente.

Para continuar con el trabajo ahora crearemos un script en Python que nos permita controlar el generador de funciones mediante comandos. Para ello primero importamos la librería pyserial, la cual nos permite comunicarnos con el puerto serial.

El script que creamos es el siguiente:

```python
import serial
import time


PUERTO = '/dev/ttyUSB0'
BAUD_RATE = 9600
```

```python

# Saltos definidos en el procedimiento
frecuencias_barrido = [1000, 2000, 5000, 10000, 20000,
    50000, 100000]

try:
    gf = serial.Serial(PUERTO, BAUD_RATE, timeout=1)
    print(f"[*] Conectado al generador en {PUERTO}.")

    comandos_iniciales = [
        "SOUR:FUNC SIN\n",
        "SOUR:VOLT 5.0\n",
        "SOUR:VOLT:OFFS 0\n",
        "OUTP ON\n"
    ]

    for cmd in comandos_iniciales:
        gf.write(cmd.encode('ascii'))
        time.sleep(0.1)

    print("[*] Configuración inicial aplicada (Senoidal, 5Vpp
        , Offset 0V).")
    print("[*] Iniciando barrido con intervención del
        operador (Opción B).")


    for freq in frecuencias_barrido:

        input(f"\n[?] Presiona ENTER para ajustar la
            frecuencia a {freq} Hz...")

        comando_freq = f"SOUR:FREQ {freq}\n"
        gf.write(comando_freq.encode('ascii'))
        print(f" ---> Generando {freq} Hz. Observa el
            osciloscopio y toma nota.")

    print("\n[*] Barrido finalizado con éxito.")

except serial.SerialException as e:
    print(f"[!] Error de puerto serie: {e}")
finally:
    if 'gf' in locals() and gf.is_open:
        gf.close()
```

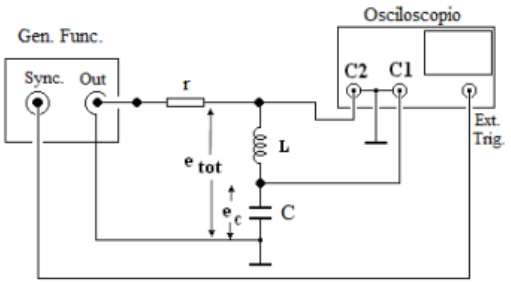


Figura 3. Esquema de la placa de pruebas



Figura 2. Configuración del generador de funciones a 9600 baudios

Al inicio del codigo definimos el puerto y la velocidad de comunicación, luego creamos una lista con las frecuencias que queremos barrer.

Luego abrimos el puerto serial y enviamos los comandos iniciales para configurar el generador de funciones.

A continuación, realizamos un barrido de frecuencias, donde el operador debe presionar ENTER para avanzar a la siguiente frecuencia.

Finalmente cerramos el puerto serial.

Cuando el codigo esta en funcionamiento, el operador debe observar la señal en el osciloscopio y tomar nota de los resultados.

Por otro lado configuramos el generador de funciones para que este tenga 9600 baudios

Utilizaremos una placa de pruebas representada por el siguiente esquema por la cual mediremos la señal y el potencial eC en el osciloscopio.

Ahora configuramos el osciloscopio para que muestre la señal de manera correcta, y cambiamos el modo al Matemático para poder observar la señal de manera más clara.

Para ello seleccionamos FFT, en el modo de ventana Hanning, y centramos la señal para luego aplicar el x10 en el eje Y.

Ahora iniciamos el barrido de frecuencias, y observamos la señal en el osciloscopio.

Vemos que a la frecuencia de 10kHz la señal se encuentra en su punto máximo.

Tocamos un poco el generador manualmente para encontrar el punto de resonancia, y vemos que se encuentra en 10.2kHz.

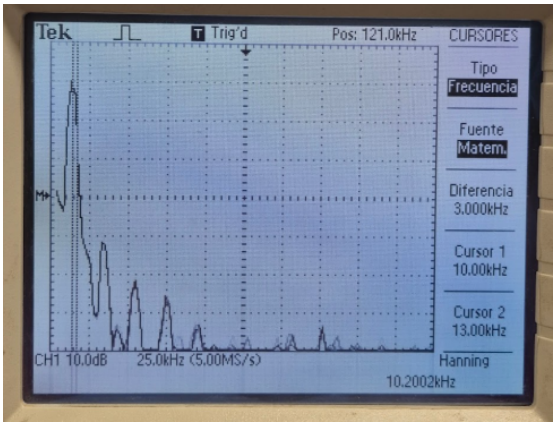


Figura 4. Configuración del osciloscopio en modo Matemático



Figura 5. Señal en el osciloscopio a 10.2kHz

Ahora modificamos el barrido para llegar a +3dB y -3dB, para que lleguemos a ese punto nuestra amplitud debe ser de 19.6dB ya que en el punto de resonancia la amplitud es de 22.6dB.

El barrido que utilizaremos es desde 8kHz hasta 14kHz, con saltos de 200Hz cuando esté entre 9kHz y 12kHz, y de 500Hz cuando esté fuera de ese rango.

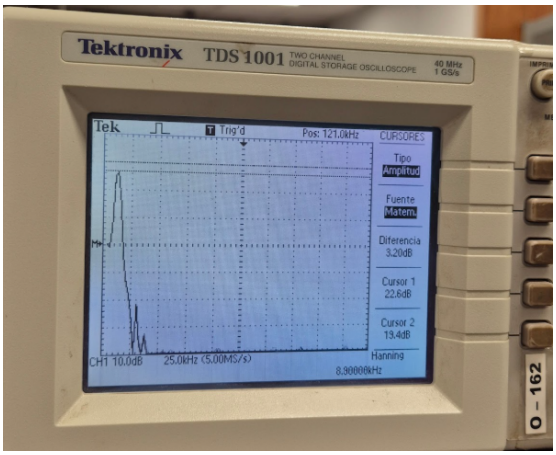


Figura 6. Señal en el osciloscopio a +3dB y -3dB

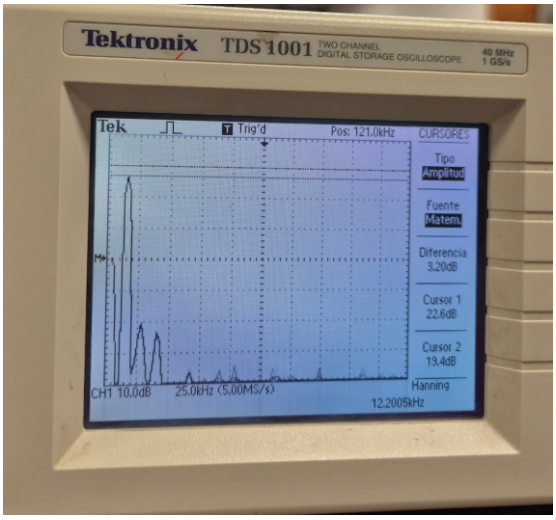


Figura 7. Señal en el osciloscopio a +3dB y -3dB

Vemos que en el punto de +3dB la frecuencia es de 12.2kHz y en el punto de -3dB la frecuencia es de 9kHz.

Como ultimo paso compararemos los resultados obtenidos con los calculados, para ello utilizaremos la siguiente formula:

$$f_r = \frac{1}{2\pi\sqrt{LC}}$$

Reemplazando los valores de L = 20mH y C = 7nF obtenemos:

$$f_r = \frac{1}{2\pi\sqrt{20mH * 7nF}} = 13{,}45kHz$$

Vemos que el valor calculado es cercano al medido de 10.2kHz, y la diferencia se debe a que los valores de L y C no son exactos, sino que tienen una tolerancia de 5 % y 10 % respectivamente.

III. Conclusiones

Referencias

[1] TP8_version_definitiva_V_01_2026_Medidas Electrónicas I, UTNFRC.